```
  def handleCommands(
      state: State,
      command: Command): ReplyEffect[Event, State] =
    (state, command) match {
      case (state: UninitializedState, command: Open) =>
                                                open(state, command)
      case (state: OpenState, command: Update) =>
                                                update(state, command)
      case (state: OpenState, command: Close) => close(state, command)
      case (_, command: Cancel)               => cancel(state, command)
      case (_, command: GetState)             => tell(state, command)
      case _                                  => invalid(state, command)
    }
  ...
}
```

The method `open(state, command)` and the rest trigger events that can change the actor's state and thus its behavior (see figure 12.2).

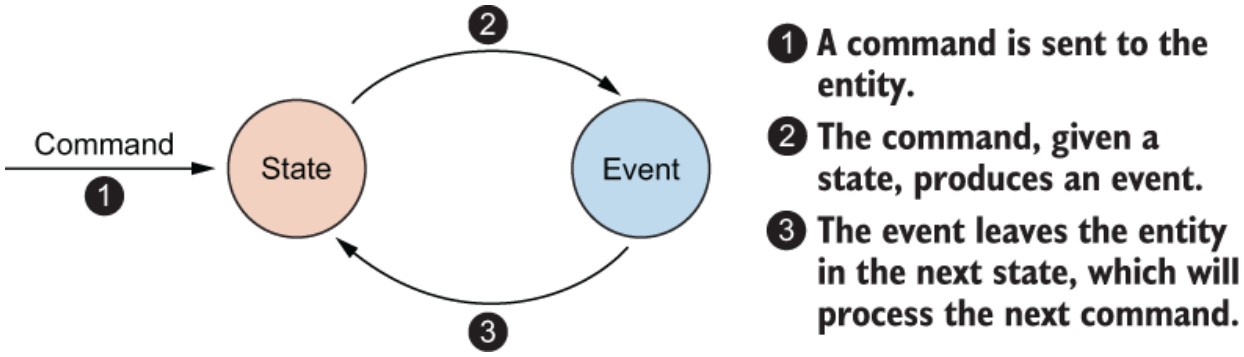


Figure 12.2 Command, state, and event interactions

The `open` method generates an `Opened` event, persists it, and then replies with `Accepted`:

```
object Market {
  ...

  private def open(
      state: State,
      command: Open): ReplyEffect[Opened, State] = {
    val opened =
```